Guía rápida: GitHub Pages

Especialización en Ciencia de Datos e IA · Universidad de Medellín

2026-10

Guía rápida: GitHub Pages

Cómo publicar tu repositorio como sitio web en 10 minutos.

¿Qué es GitHub Pages?

Un servicio **gratuito** de GitHub que convierte los archivos de tu repositorio en un sitio web público. Los `README.md` que ya tienes se renderizan como páginas HTML con tema. No necesitas conocer HTML/CSS.

Caso típico: publicar la documentación de tu proyecto final para que un empleador o un profesor pueda navegarla sin clonar el repo.

URL que obtienes: `https://TU-USUARIO.github.io/TU-REPO/`

¿Qué necesitas?

1. Una cuenta de GitHub.
2. Un repositorio tuyo (público, o privado si tienes plan Pro).
3. Al menos un `README.md` en el repo.

Eso es todo. **No necesitas conocer Jekyll, HTML ni CSS para empezar.**

Las 3 opciones de origen (y por qué elegimos `main /`)

GitHub Pages puede servir desde tres lugares distintos:

Opción	Qué significa	Ramas que tienes al final
main /	Sirve desde la raíz de main	Solo `main`
main /docs	Sirve desde la subcarpeta `docs/` de main	Solo `main`
gh-pages	Rama dedicada	`main` + `gh-pages`

Recomendación: usa `main /`. Es la más simple: una sola rama que mantener, y cada `git push` a `main` actualiza el sitio automático en ~1 minuto.

Paso 1 — Añade `_config.yml` a tu repo

En la raíz de tu repositorio, crea un archivo llamado `_config.yml` con este contenido mínimo:

```
title: Mi Proyecto
description: Breve descripción
theme: jekyll-theme-cayman
```

Qué significa cada línea:

- **`title`**: aparece en la pestaña del navegador y en el header del sitio.
- **`description`**: subtítulo bajo el título.
- **`theme`**: tema visual (lista oficial en https://pages.github.com/themes/).

Si tu repo tiene código Python, notebooks o datos

Añade un `exclude:` para que Jekyll ignore los archivos que no son contenido web:

```yaml
title: Mi Proyecto
description: Breve descripción
theme: jekyll-theme-cayman

exclude:
  - "**/*.ipynb"       # notebooks
  - "**/*.py"          # código Python
  - "**/*.csv"         # datos
  - "**/data/"         # carpetas de datos
  - "**/models/"       # modelos entrenados
  - uv.lock
  - pyproject.toml
  - Makefile
```

Si quieres sidebar + prev/next (estilo libro navegable)

Reemplaza el `theme:` por un tema de documentación como `just-the-docs`:

```yaml
title: Mi Proyecto
remote_theme: just-the-docs/just-the-docs
plugins:
  - jekyll-remote-theme
search_enabled: true
```

Nota: `just-the-docs` requiere que cada `.md` tenga frontmatter (`title`, `parent`, `nav_order`) para aparecer bien organizado en el sidebar. Ver tutorial 01c del repo del curso para el detalle completo.

Paso 2 — Commitea y pushea el archivo

Siguiendo el flujo normal de git (**nunca** directo en main):

```bash
git switch -c feat/github-pages
git add _config.yml
git commit -m "feat: configurar GitHub Pages"
git push -u origin feat/github-pages
```

Abre un **Pull Request** en github.com y mergealo a `main`.

Paso 3 — Activa Pages en Settings

En tu repositorio en github.com:

1. Click en **Settings** (pestaña superior derecha del repo).
2. En el menú lateral izquierdo, busca y click en **Pages**.
3. En **Source**, selecciona: **Deploy from a branch**.
4. En **Branch**, elige:
   - **Branch:** `main`
   - **Folder:** `/ (root)`
5. Click **Save**.

Es un proceso de 30 segundos.

Paso 4 — Espera 1 minuto y verifica

GitHub construye tu sitio automático. En la pestaña **Actions** del repo verás un workflow llamado `pages-build-deployment` ejecutándose.

Cuando termine (~1 min), vuelve a **Settings** → **Pages**. Verás arriba:

> **Your site is live at** `https://TU-USUARIO.github.io/TU-REPO/`

Click en ese link. Deberías ver tu `README.md` renderizado como página web con el tema que elegiste.

Cómo se actualiza el sitio

Cada vez que haces push a `main`, GitHub reconstruye el sitio automático en ~1 minuto. **No tienes que hacer nada más.**

Si editaste un `.md` y en 2 minutos no ves el cambio en el sitio, revisa la pestaña **Actions** para ver si el workflow `pages-build-deployment` falló.

Problemas comunes

“404 Page not found”

- Verifica que `_config.yml` está en la **raíz** del repo (no en una subcarpeta).
- Verifica que la URL incluye el nombre del repo (`/TU-REPO/`), con guiones y en minúsculas.
- La primera vez puede tardar hasta 10 minutos; espera un poco más.

La página se ve rota (sin estilos)

- Verifica que el nombre del `theme` es correcto (lista en pages.github.com/themes/).
- Revisa en **Actions** si hubo un error de build.

Mis notebooks no se renderizan como páginas

- Es intencional: Jekyll no soporta `.ipynb` nativamente. Enlázalos a **nbviewer.org** o directamente a github.com (que los renderiza aceptablemente).

Un `.md` importante no aparece en el sitio

- Chequea que no esté dentro de una carpeta excluida en `_config.yml`.
- Chequea que el nombre del archivo **no empiece con** `_` (Jekyll ignora cosas que empiezan con guión bajo por convención).

“Invalid YAML” en Actions

- Tu `_config.yml` tiene mala indentación o un carácter raro.
- La indentación de YAML es con **espacios**, no tabs. Dos espacios por nivel.
- Comillas: todas dobles (`"`) o todas simples (`'`), consistente.

Ejemplo vivo

El repositorio del curso usa esta misma configuración:

- **Sitio:** https://dpalacioj.github.io/data-projects-lab/
- **Código fuente:** https://github.com/dpalacioj/data-projects-lab

Explora cómo se ve y revisa el `_config.yml` del repo si quieres copiar la configuración completa (incluye `just-the-docs` con sidebar jerárquico y search).

Resumen en 5 pasos

1. Crea `_config.yml` en la raíz de tu repo (3 líneas mínimo).
2. Commit + Pull Request + merge a `main`.
3. **Settings → Pages → Source: Deploy from a branch → main / (root) → Save**.
4. Espera 1 min, visita `https://TU-USUARIO.github.io/TU-REPO/`.
5. Cada push a `main` actualiza el sitio automático.

Una sola rama (`main`), un solo archivo nuevo (`_config.yml`), 5 clicks en Settings.